

Placement Empowerment Program
Cloud Computing and DevOps Centre

**Create a Simple Backup Script Create a script that
backups up your entire Git repository to a local folder
daily**

Name: Neithal Pillai

Department : IT

Introduction and Overview

In this Proof of Concept (PoC), we explore how to automate the deployment of a static website on a local server using Git and shell scripting. This process ensures that any updates pushed to the repository are automatically reflected on the server without manual intervention. Automating this workflow simplifies the deployment process, reduces errors, and ensures consistency in website updates.

Objective The goal of this project is to:

1. Set up a local web server for hosting static files.
2. Configure Git hooks to automate deployment.
3. Create a script that updates the website whenever changes are pushed.
4. Ensure seamless synchronization between the repository and the hosted files.
5. Test and verify that the automation works correctly.

Importance of Automated Deployment Automating the deployment of a static website is crucial for efficiency and reliability. It offers several benefits, including:

- **Time Efficiency:** Eliminates the need for manual file uploads after every change.
- **Consistency:** Ensures all updates are correctly reflected on the server without discrepancies.
- **Error Reduction:** Minimizes the risk of human error in deployment.
- **Scalability:** Makes it easier to manage multiple website updates efficiently.

Steps for Execution

Step 1: Set Up a Local Web Server To begin, we set up a local web server using Apache or a lightweight alternative like Python's built-in HTTP server. This server hosts the static website files, ensuring they are accessible through a local URL. Once installed, the document root is set to the folder containing the static files to be served.

```
MINGW64:/c/Users/neith/Desktop/CLOUD/Apache24/htdocs
--[no-]also-filter-submodules      apply partial clone filters to submodules
--[no-]remote-submodules          any cloned submodules will use their remote-tracking b
branch
--[no-]sparse                     initialize sparse-checkout file to include only files
at root
--[no-]bundle-uri <uri>          a URI for downloading bundles before fetching from ori
gin remote

neith@Neithals-Laptop MINGW64 ~/Desktop/CLOUD/Apache24/htdocs
$ git clone https://github.com/Neithal07/POC-Projects.git
Cloning into 'POC-Projects'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 15 (delta 1), reused 15 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 125.35 KiB | 777.00 KiB/s, done.
Resolving deltas: 100% (1/1), done.

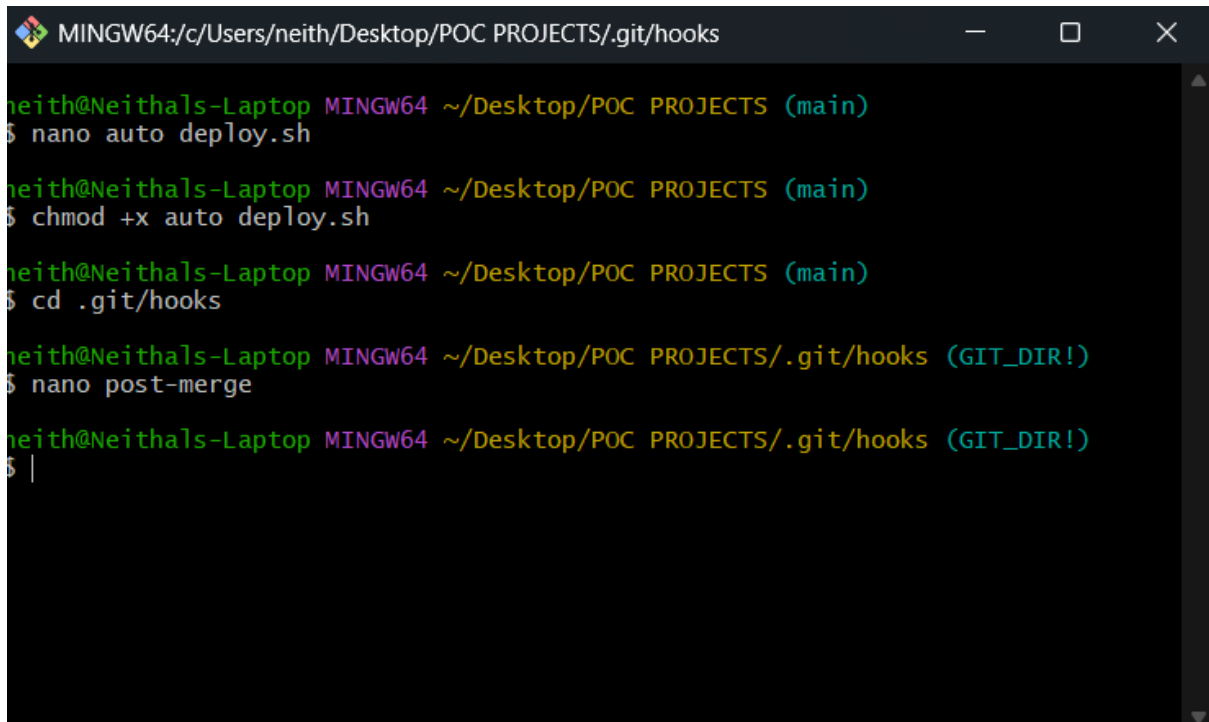
neith@Neithals-Laptop MINGW64 ~/Desktop/CLOUD/Apache24/htdocs
$
```

Step 2: Initialize a Git Repository Inside the website's project folder, a Git repository is initialized to track changes. The repository is linked to a remote repository (e.g., GitHub or GitLab), where updates will be pushed. Essential files like HTML, CSS, and JavaScript are added and committed to ensure version control is in place.

```
MINGW64:/c/Users/neith/Desktop/POC PROJECTS
[1/2] auto Modified
#!/bin/bash
cd /c/Apache24/htdocs/your-repo # Navigate to your website directory
git pull origin main # Fetch latest changes
echo "Website updated successfully!"

AG Help      AO Write Out  AF Where Is  AK Cut       AT Execute   AC Location
AX Close     AR Read File  A\ Replace   AU Paste     AJ Justify   A/ Go To Line
```

Step 3: Configure a Git Hook for Automation To automate deployment, a Git post-receive hook is created inside the repository's hooks directory. This script listens for new pushes and triggers an update process to sync the latest changes to the website's hosted directory. The hook ensures that whenever new content is pushed, the web server serves the updated files automatically.

A terminal window titled 'MINGW64:/c/Users/neith/Desktop/POC PROJECTS/.git/hooks' shows a series of commands being executed. The user is in the 'main' branch of a repository. They create a file named 'auto deploy.sh' using 'nano', set its permissions to executable with 'chmod +x', move to the '.git/hooks' directory, and create a file named 'post-merge'.

```
MINGW64:/c/Users/neith/Desktop/POC PROJECTS/.git/hooks
neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ nano auto deploy.sh
neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ chmod +x auto deploy.sh
neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ cd .git/hooks
neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS/.git/hooks (GIT_DIR!)
$ nano post-merge
neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS/.git/hooks (GIT_DIR!)
$ |
```

Step 4: Create and Test the Deployment Script A shell script is developed to pull the latest changes from the Git repository and update the web server's document root. The script is tested to ensure that whenever a push is made, the website updates seamlessly without requiring manual intervention. Permissions are adjusted to allow the script to execute automatically.

```
MINGW64:/c/Users/neith/Desktop/POC PROJECTS
he next time Git touches it

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ git commit -m "Updated website content"
[main 83834ae] Updated website content
 3 files changed, 45 insertions(+)
 create mode 100644 auto
 create mode 100644 deploy.sh
 create mode 100644 monitor_logs.sh

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 1.00 KiB | 512.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/Neithal07/POC-Projects.git
   2bfa475..83834ae  main -> main

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$
```

Step 5: Verify and Monitor the Deployment Process After setting up automation, a series of tests are performed to confirm that changes pushed to the repository are correctly reflected on the local web server. Logs and error messages are monitored to troubleshoot any issues. Finally, periodic maintenance checks are scheduled to ensure the automated deployment remains functional.

Neithal07 Updated website content 83834ae · 1 minute ago 5 Commits		
 .gitignore	Ignore log files	6 hours ago
 @echo off.bat	Added a new html file to demonstrate branch testing	last week
 Mywebsite.html	Added a new html file to demonstrate branch testing	last week
 Mywebsite.txt	Added a new html file to demonstrate branch testing	last week
 README.md	Added a new html file to demonstrate branch testing	last week
 Slide1.PNG	Added a new html file to demonstrate branch testing	last week
 Testbranch.html	Added a new html file to demonstrate branch testing	last week
 auto	Updated website content	1 minute ago
 backup_git.bat	Added a new html file to demonstrate branch testing	last week
 deploy.sh	Updated website content	1 minute ago
 index.html	Added a new html file to demonstrate branch testing	last week
 monitor_logs.sh	Updated website content	1 minute ago

Conclusion By following these steps, we successfully automated the deployment of a static website on a local server. The use of Git hooks and shell scripting streamlined the process, making it efficient and error-free. This setup ensures that any updates to the website are instantly reflected on the server, providing a seamless workflow for local development and testing.

What I Learned from This Process

- **Automated Deployment:** Implementing Git hooks to update the web server automatically.
- **Git Workflow:** Managing version control and repository synchronization.
- **Shell Scripting:** Writing and executing scripts to streamline deployment tasks.
- **Server Configuration:** Setting up a local web server for static file hosting.
- **Debugging & Monitoring:** Troubleshooting automation issues to ensure reliability.